COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	August 04, 2026	**DATE SUBMITTED:**	August 04, 2026 & August 06, 2026
NAME:	Niña Denise M. Santiago	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

HANDS-ON ACTIVITY NO. 5.1 **Queues**

1. PROCEDURE OUTPUT

ILO A:

Source Code:

ILOA > mainA.cpp > ...

```cpp
#include<iostream>
#include<queue>

void displayAll(std::queue<char> copyQueue);

void isEmpty(std::queue<char> emptyQueue);

int main(){
    // Create an object
    std::queue<char> myQ;

    std::cout << "~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"<< std::endl;

    // Empty
    isEmpty(myQ);

    std::cout << "~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"<< std::endl;

    // Pushes Characters in the Queue
    myQ.push('N');
    myQ.push('I');
    myQ.push('N');
    myQ.push('Y');
    myQ.push('A');
    // Displays Queue
    displayAll(myQ);

```

```
28      // Organizes Output
29      std::cout << "~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"<< std::endl;
30
31      // Empty
32      isEmpty(myQ);
33
34      std::cout << "~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"<< std::endl;
35
36      // Size
37      std::cout << "Queue size: " << myQ.size() << "\n";
38
39      // Organizes Output
40      std::cout << "~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"<< std::endl;
41
42      // Front
43      std::cout << "Front of the Queue: " << myQ.front() << "\n";
44
45      // Organizes Output
46      std::cout << "~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"<< std::endl;
47
48      // Back
49      std::cout << "Back of the Queue: " << myQ.back() << "\n";
50
51      // Push
52      myQ.push('H');
53
54      // Display
55      displayAll(myQ);
56
57      // Organizes Output
58      std::cout << "~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"<< std::endl;
59
60      // Pop
61      std::cout << "Character to be popped: " << myQ.front() << "\n";
62      myQ.pop();
63
64      // Display
65      displayAll(myQ);
66
67      // Organizes Output
68      std::cout << "~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~"<< std::endl;
69
```

```
70
71
72      return 0;
73  }
74
75  void displayAll(std::queue<char> copyQueue){
76      // Creates a copy of the queue that was passed
77      std::queue<char> c = copyQueue; // Temporary Queue
78
79      while(!c.empty()){
80          std::cout << "" << c.front();
81          c.pop();
82      }
83      std::cout << "\n";
84  }
85
86  void isEmpty(std::queue<char> emptyQueue){
87          // Empty
88      if (emptyQueue.empty()){
89          std::cout << "Queue is Empty\n";
90      }else{
91          std::cout << "Queue is NOT Empty\n";
92      }
93  }
```

Output:

```
Queue is Empty
NINYA
Queue is NOT Empty
Queue size: 5
Front of the Queue: N
Back of the Queue: A
NINYAH
Character to be popped: N
INYAH
```

Remarks:

Using STL for queues is just a straightforward approach like Linked lists and Stacks. Using the keywords provided by the library, it is easier to use the commands to code in main. Just like shown in the snippet, most of these commands are to call the built-in functions for queues. Additional functions were only made for displayAll and isEmpty for code reusability and easier understanding of the code. Overall, ILO A just introduces the concept of what and how queues are used in programming.

ILO B:

Header File:

ILOB > C queueLL.h > ...

```
#ifndef QUEUELL_H
#define QUEUELL_H

// Create a Class for the Node
template <typename T>
class qNode{
    public:
    T data;
    qNode<T>* next;
};

// Creates the New Node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

// Enqueue Operation
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    // Create new node
    qNode<T>* newNode = new_node(newData);

    // Inserting into an empty queue
    if((*frontPtr)==nullptr && (*backPtr)==nullptr){ // Checks if Queue is Empty
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }

    // Inserting into a non-empty queue
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}

// Dequeue Operation
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    // Adds a Check if the queue is already empty
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        std::cout << "Queue is EMPTY" << std::endl;
    }

```

```
46      // Create a Temporary node
47      qNode<T>* deleteNode = nullptr; // Temporary Variable
48      deleteNode = (*frontPtr);
49
50      // Check if the queue has only one node
51      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
52          (*frontPtr) = nullptr;
53          (*backPtr) = nullptr;
54          delete deleteNode;
55          return;
56      }
57
58      // Delete the Node
59      (*frontPtr) = (*frontPtr)->next;
60      deleteNode->next = nullptr;
61      delete deleteNode;
62  }
63
64  // Checks the Front
65  template <typename T>
66  void frontCheck(qNode<T>** frontPtr, qNode<T>** backPtr){
67      // Checks if the queue is empty
68      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
69      std::cout << "Queue is EMPTY" << std::endl;
70      }
71
72      // Access the front node data/pointer
73      qNode<T>* front = *frontPtr;
74      std::cout << "Front Element: " << front->data << std::endl;
75  }
76
77  // Checks the Back
78  template <typename T>
79  void backCheck(qNode<T>** frontPtr, qNode<T>** backPtr){
80      // Checks if the queue is empty
81      if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
82      std::cout << "Queue is EMPTY" << std::endl;
83      }
84
85      // Access the front node data/pointer
86      qNode<T>* back = *backPtr;
87      std::cout << "Back Element: " << back->data << std::endl;
88  }
89
```

```
89
90
91  // Display All
92  template <typename T>
93  void displayAll(qNode<T> **frontPtr, qNode<T> **backPtr){
94      // Checks if queue is Empty
95          if((*frontPtr) == nullptr && (*backPtr) == nullptr){
96              std::cout << "The queue is empty!" << std::endl;
97              return;
98          }
99
100     // Traverse through the queue to Display All
101     std::cout << "Displaying the Entire queue: " << std::endl;
102     while ((*frontPtr) != nullptr){
103         std::cout << (*frontPtr)->data << " ";
104         (*frontPtr) = (*frontPtr)->next;
105     }
106     std::cout<<std::endl;
107 }
108
109 #endif
```

Main File:

ILOB > mainB.cpp > ...

```
1   #include<iostream>
2   #include"queueLL.h"
3
4   int main(){
5       qNode<char>* front = nullptr;
6       qNode<char>* back = nullptr;
7
8       // For Organization Purposes
9       std::cout << "========================================" << std::endl;
10
11      // Enqueue Operation Testing:
12      enqueue('N', &front, &back);
13      std::cout << front->data << " " << back->data << std::endl;
14      enqueue('I', &front, &back);
15      std::cout << front->data << " " << back->data << std::endl;
16      enqueue('N', &front, &back);
17      std::cout << front->data << " " << back->data << std::endl;
18      enqueue('S', &front, &back);
19      std::cout << front->data << " " << back->data << std::endl;
20
```

```
21      // For Organization Purposes
22      std::cout << "========================================" << std::endl;
23
24      // Front Check Operation Testing:
25      frontCheck(&front, &back);
26
27      // For Organization Purposes
28      std::cout << "========================================" << std::endl;
29
30      // Back Check Operation Testing:
31      backCheck(&front, &back);
32
33      // For Organization Purposes
34      std::cout << "========================================" << std::endl;
35
36      // Dequeue Operation Testing:
37      dequeue(&front, &back);
38      std::cout << front->data << " " << back->data << std::endl;
39      dequeue(&front, &back);
40      std::cout << front->data << " " << back->data << std::endl;
41      dequeue(&front, &back);
42      std::cout << front->data << " " << back->data << std::endl;
43      dequeue(&front, &back);
44      dequeue(&front, &back);
45
46
47      return 0;
48  }
```

Output:

```
========================================
N N
N I
N N
N S
========================================
Front Element: N
========================================
Back Element: S
========================================
I S
N S
S S
Queue is EMPTY
```

Remarks:

ILO B required creating a custom header file and manually implementing queue operations using a linked list. Unlike using STL in ILO A, which hides the complexity, this approach showed me how queues work internally by managing nodes, pointers, and memory directly. It emphasized to me the importance of maintaining both front and back pointers to preserve the FIFO behavior. While more challenging, this method gave me a clearer understanding of how queues differ from stacks and linked lists, and strengthened my grasp of dynamic memory management.

ILO B.2: Array-Based Implementation (Active Coding with Sir)

Header File:

ILOB > C queueArr.h > QueueArr(size_t)

```h
#ifndef QUEUEARRAY_H
#define QUEUEARRAY_H

#include <iostream>

template <typename T>
class QueueArr {
private:
    T* q_array;      // pointer to array
    size_t q_capacity;    // max capacity
    size_t q_size;        // current size
    int q_front;       // index of front
    int q_back;        // index of back

public:
    // Constructor
    QueueArr(size_t capacity);
    ~QueueArr();

    bool Empty();
    bool Full();

    size_t Size();

    void Clear();

    T Front();
    T Back();

    void Enqueue(T newData);
    void Dequeue();

    QueueArr(const QueueArr& other); // Copy Constructor
    QueueArr& operator=(const QueueArr& other); // Copy Assignment Operator

    void DisplayAll();
};

```

```cpp
// Constructor
template <typename T>
QueueArr<T>::QueueArr(size_t capacity) {
    // Define the capacity of the array
    q_capacity = capacity;
    // Set the size to 0
    q_size = 0;
    // Set the front and back to 0 and -1 respectively
    q_front = 0;
    q_back = -1;
    // Allocate a new space for the array
    q_array = new T[q_capacity];
}

// Destructor
template <typename T>
QueueArr<T>::~QueueArr() {
    delete[] q_array;
    std::cout << "Destructor was Called" << std::endl;
}

// Is Empty
template <typename T>
bool QueueArr<T>::Empty() {
    return q_size == 0;
}

// Is Full
template <typename T>
bool QueueArr<T>::Full() {
    return q_size == q_capacity;
}

// Return size
template <typename T>
size_t QueueArr<T>::Size() {
    return q_size;
}
```

```cpp
// Clear Queue
template <typename T>
void QueueArr<T>::Clear() {
    // Queue size set to 0
    q_size = 0;
    // Restores the front and back to their initial values
    q_front = 0;
    q_back = -1;
    return;
}

// Get front element
template <typename T>
T QueueArr<T>::Front() {
    // Check if the queue is empty
    if (Empty()){
        return T(); // Return default value of T if empty
    }

    // Return the front element
    return q_array[q_front];
}

// Get back element
template <typename T>
T QueueArr<T>::Back() {
    // Check if the queue is empty
    if (Empty()){
        return T(); // Return default value of T if empty
    }
    return q_array[q_back];
}

```

```cpp

 // Enqueueing Function
 template <typename T>
 void QueueArr<T>::Enqueue(T newData) {
     // Check if the queue is full
     if (Full()) {
         std::cout << "Queue is full!" << std::endl;
         return;
     }
     // Increment the back index (rear) in a circular manner
     q_back = (q_back + 1) % q_capacity;
     // Add the new value to the back of the queue
     q_array[q_back] = newData;
     // Increment the size of the queue
     q_size++;
 }

```

```cpp
// Dequeueing Function
template <typename T>
void QueueArr<T>::Dequeue() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is empty!" << std::endl;
        return;
    }
    // Increment the front index (front) in a circular manner
    q_front = (q_front + 1) % q_capacity;
    // Decrement the size of the queue
    q_size--;
    // When the queue is empty...
    if (q_size == 0) {
        Clear(); // Reset the queue to its initial state
        return;
    }
}

// Copy Constructor
template <typename T>
QueueArr<T>::QueueArr(const QueueArr& other) {
    // Copy the capacity and size from the other queue
    q_capacity = other.q_capacity;
    q_size = other.q_size;
    // Copy the front and back indices
    q_front = other.q_front;
    q_back = other.q_back;
    // Allocate new memory for the array and copy the elements
    q_array = new T[q_capacity];
    for (size_t i = 0; i < q_capacity; ++i) {
        q_array[i] = other.q_array[i];
    }
}

```

```cpp
// Copy Assignment Operator
template <typename T>
QueueArr<T>& QueueArr<T>::operator=(const QueueArr& other) {
    // Check for self-assignment
    if (this != &other) {
        // Delete the existing array
        delete[] q_array;
        // Copy the capacity and size from the other queue
        q_capacity = other.q_capacity;
        q_size = other.q_size;
        // Copy the front and back indices
        q_front = other.q_front;
        q_back = other.q_back;
        // Allocate new memory for the array and copy the elements
        q_array = new T[q_capacity];
        for (size_t i = 0; i < q_capacity; ++i) {
            q_array[i] = other.q_array[i];
        }
    }
    return *this; // Return the current object
}
```

```cpp
// Display All Function
template <typename T>
void QueueArr<T>::DisplayAll() {
    // Check if the queue is empty
    if (Empty()) {
        std::cout << "Queue is empty!" << std::endl;
        return;
    }
    // Display all elements in the queue
    std::cout << "Queue elements: ";
    for (size_t i = 0; i < q_size; ++i) {
        size_t index = (q_front + i) % q_capacity;
        std::cout << q_array[index] << " ";
    }
}

#endif
```

Main File: (int and char)

ILOB > mainBArr.cpp > ...

```cpp
#include <iostream>
#include "queueLL.h"
#include "queueARR.h"

int main() {

    // Create an Object
    QueueArr<int> q(6);

    std::cout << "=====================================================================" << std::endl;

    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Enqueue(4);
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Enqueue(5);
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Enqueue(6);
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Enqueue(7);
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;

    std::cout << "=====================================================================" << std::endl;

    q.DisplayAll();

    std::cout << "\n=====================================================================" << std::endl;

    q.Dequeue();
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Dequeue();
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Dequeue();
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Clear();
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Dequeue();

    std::cout << "=====================================================================" << std::endl;

    q.DisplayAll();

    std::cout << "=====================================================================" << std::endl;

    return 0;
}
```

ILOB > mainBArr.cpp > ...

```cpp
#include <iostream>
#include "queueLL.h"
#include "queueARR.h"

int main() {

    // Create an Object
    QueueArr<char> q(6);

    std::cout << "=====================================================================" << std::endl;

    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Enqueue('Y');
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Enqueue('A');
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Enqueue('N');
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Enqueue('I');
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;

    std::cout << "=====================================================================" << std::endl;

    q.DisplayAll();

    std::cout << "\n=====================================================================" << std::endl;

    q.Dequeue();
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Dequeue();
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Dequeue();
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Clear();
    std::cout<<"q_front: " << q.Front() <<" q_back: " <<q.Back() << " q_size: " <<q.Size()<<std::endl;
    q.Dequeue();

    std::cout << "=====================================================================" << std::endl;

    q.DisplayAll();

    std::cout << "=====================================================================" << std::endl;

    return 0;
}
```

Output:

```
==================================================================
q_front: 0 q_back: 0 q_size: 0
q_front: 4 q_back: 4 q_size: 1
q_front: 4 q_back: 5 q_size: 2
q_front: 4 q_back: 6 q_size: 3
q_front: 4 q_back: 7 q_size: 4
==================================================================
Queue elements: 4 5 6 7
==================================================================
q_front: 5 q_back: 7 q_size: 3
q_front: 6 q_back: 7 q_size: 2
q_front: 7 q_back: 7 q_size: 1
q_front: 0 q_back: 0 q_size: 0
Queue is empty!
==================================================================
Queue is empty!
==================================================================
Destructor was Called
```

Integer Application

```
==================================================================
q_front:  q_back:  q_size: 0
q_front: Y q_back: Y q_size: 1
q_front: Y q_back: A q_size: 2
q_front: Y q_back: N q_size: 3
q_front: Y q_back: I q_size: 4
==================================================================
Queue elements: Y A N I
==================================================================
q_front: A q_back: I q_size: 3
q_front: N q_back: I q_size: 2
q_front: I q_back: I q_size: 1
q_front:  q_back:  q_size: 0
Queue is empty!
==================================================================
Queue is empty!
==================================================================
Destructor was Called
```

Character Application

Code Analysis:

This program shows a queue using an array implementation and handles it in a circular way for a more realistic approach of a queue. The constructor sets up the array with a fixed capacity, initializes the size to zero, and sets the front and back positions. The destructor deletes the array when the object is destroyed. The Empty() and Full() functions check if the queue has no elements or if it's already filled to its capacity. Size() just returns how

many items are currently stored. Clear() resets everything back to the starting state. When we call Front() or Back(), the code gives us the first or last element, but if the queue is empty it returns a default value of the type. Adding data is done with Enqueue(), which moves the back index forward in a circular way and places the new item there, then increases the size. Removing data is done with Dequeue(), which moves the front index forward in a circular way and decreases the size, and if the queue becomes empty it resets everything. The copy constructor and assignment operator make sure that if we copy one queue to another, the data is duplicated properly with new memory. Finally, the DisplayAll() function loops through the queue from front to back and prints all the elements in order, which is useful for checking what's inside. Overall, the code shows clearly how a circular array queue works step by step.

ILO B.2: Array-Based Implementation (Asynchronous made by me)

Header File:

ILOB_2 > C queueARR.h > ...

```h
#ifndef QUEUEARRAY_H
#define QUEUEARRAY_H

#include <iostream>

class Queue {
private:
    int* q_array;       // pointer to array
    int q_capacity;     // max capacity
    int q_size;         // current size
    int q_front;        // index of front
    int q_back;         // index of back

public:
    // Constructor
    Queue(int capacity = 10) {
        q_capacity = capacity;
        q_array = new int[q_capacity];
        q_size = 0;
        q_front = 0;
        q_back = -1;
    }

    // Destructor
    ~Queue() {
        delete[] q_array;
    }

    // Copy Constructor
    Queue(const Queue& queueToCopy) {
        q_capacity = queueToCopy.q_capacity;
        q_size = queueToCopy.q_size;
        q_front = queueToCopy.q_front;
        q_back = queueToCopy.q_back;
        q_array = new int[q_capacity];
        for (int i = 0; i < q_capacity; i++) {
            q_array[i] = queueToCopy.q_array[i];
        }
    }

```

```
41      // Copy Assignment Operator
42      Queue& operator=(const Queue& queueToCopy) {
43          if (this != &queueToCopy) {
44              delete[] q_array;
45              q_capacity = queueToCopy.q_capacity;
46              q_size = queueToCopy.q_size;
47              q_front = queueToCopy.q_front;
48              q_back = queueToCopy.q_back;
49              q_array = new int[q_capacity];
50              for (int i = 0; i < q_capacity; i++) {
51                  q_array[i] = queueToCopy.q_array[i];
52              }
53          }
54          return *this;
55      }
56
57      // Check if empty
58      bool Empty() {
59          return q_size == 0;
60      }
61
62      // Return size
63      int Size() {
64          return q_size;
65      }
66
67      // Clear queue
68      void Clear() {
69          q_size = 0;
70          q_front = 0;
71          q_back = -1;
72      }
73
74      // Get front element
75      int Front() {
76          if (Empty()) throw std::runtime_error("Queue is empty!");
77          return q_array[q_front];
78      }
79
80      // Get back element
81      int Back() {
82          if (Empty()) throw std::runtime_error("Queue is empty!");
83          return q_array[q_back];
84      }
85
86      // Enqueue
87      void Enqueue(int value) {
88          if (q_size == q_capacity) throw std::runtime_error("Queue is full!");
89          q_back = (q_back + 1) % q_capacity;
90          q_array[q_back] = value;
91          q_size++;
92      }
93
```

```
94      // Dequeue
95      void Dequeue() {
96          if (Empty()) throw std::runtime_error("Queue is empty!");
97          q_front = (q_front + 1) % q_capacity;
98          q_size--;
99      }
100
101     // Displays queue
102     void Display() {
103         if (Empty()) {
104             std::cout << "Queue is empty\n";
105             return;
106         }
107         std::cout << "Queue: ";
108         for (int i = 0; i < q_size; i++) {
109             int index = (q_front + i) % q_capacity;
110             std::cout << q_array[index] << " ";
111         }
112         std::cout << std::endl;
113     }
114 };
115
116 #endif
```

Main File:

ILOB_2 > mainb_2.cpp > ...

```cpp
#include <iostream>
#include "queueARR.h"

int main() {
    Queue q(5);

    std::cout << "==============================" << std::endl;
    std::cout << "Is the Queue Empty? " << q.Empty() << std::endl;
    std::cout << "==============================" << std::endl;

    q.Enqueue(05);
    q.Enqueue(01);
    q.Enqueue(26);
    q.Display();

    std::cout << "==============================" << std::endl;
    std::cout << "Front Element: " << q.Front() << std::endl;
    std::cout << "Back Element: " << q.Back() << std::endl;
    std::cout << "Size of Array: " << q.Size() << std::endl;
    std::cout << "==============================" << std::endl;

    q.Dequeue();
    q.Display();

    std::cout << "==============================" << std::endl;

    q.Enqueue(8);
    q.Enqueue(10);
    q.Display();

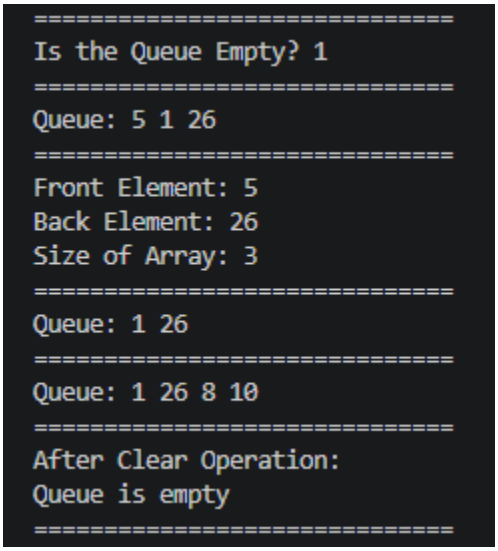
    std::cout << "==============================" << std::endl;

    q.Clear();
    std::cout << "After Clear Operation: " << std::endl;
    q.Display();

    std::cout << "==============================" << std::endl;

    return 0;
}
```

Output:



Code Analysis:

For my ILO B.2, it focuses on how to implement an array-based queue. In the header file, I created a Queue class that uses a dynamically allocated array to store elements. This class keeps track of important variables such as the capacity of the array, the current size of the queue, and the positions of the front and back. These variables are important because they allow the queue to behave like a cycle, meaning when the back reaches the end of the array, it loops back to the beginning. The member functions in the header file directly represent queue operations: Enqueue() adds an element to the back, Dequeue() removes the element at the front, Front() and Back() lets me check the first and last elements int the active queue, Empty() and Size() give the current state of the queue, and Clear() resets everything. I also included a Display() function to show the contents of the queue, which helped me visualize the operations during testing.

In the main file, I tested each function step by step. First, I checked if the queue was empty (which really is empty), then I added several elements using Enqueue(). The output showed the queue growing in order, which confirmed that insertion worked correctly. I then used Front() and Back() to check the first and last elements, and the results matched what was needed. When I called Dequeue(), the front element was removed, and the display showed the updated queue. I also tested adding more elements after a dequeue to confirm that the circular behavior of the queue worked, and the output shows the queue still functions properly. Finally, I used Clear() to reset the queue, and the output showed it was empty again.

I tried implementing the array-based approach for queues using Characters

ILO B.2: Another Version

Header File:

ILOB_2 > C queueARRchar.h > ...

```h
#ifndef QUEUEARRAYCHAR_H
#define QUEUEARRAYCHAR_H

#include <iostream>

class QueueChar {
private:
    char* q_array;      // pointer to array
    int q_capacity;     // max capacity
    int q_size;         // current size
    int q_front;        // index of front
    int q_back;         // index of back

public:
    // Constructor
    QueueChar(int capacity = 10) {
        q_capacity = capacity;
        q_array = new char[q_capacity];
        q_size = 0;
        q_front = 0;
        q_back = -1;
    }

    // Destructor
    ~QueueChar() {
        delete[] q_array;
    }

    // Check if empty
    bool Empty() {
        return q_size == 0;
    }
```

```
33
34      // Return size
35      int Size() {
36          return q_size;
37      }
38
39      // Clear queue
40      void Clear() {
41          q_size = 0;
42          q_front = 0;
43          q_back = -1;
44      }
45
46      // Get front element
47      char Front() {
48          if (Empty()) throw std::runtime_error("Queue is empty!");
49          return q_array[q_front];
50      }
51
52      // Get back element
53      char Back() {
54          if (Empty()) throw std::runtime_error("Queue is empty!");
55          return q_array[q_back];
56      }
57
58      // Enqueue
59      void Enqueue(char value) {
60          if (q_size == q_capacity) throw std::runtime_error("Queue is full!");
61          q_back = (q_back + 1) % q_capacity;
62          q_array[q_back] = value;
63          q_size++;
64      }
65
66      // Dequeue
67      void Dequeue() {
68          if (Empty()) throw std::runtime_error("Queue is empty!");
69          q_front = (q_front + 1) % q_capacity;
70          q_size--;
71      }
72
73      // Display queue
74      void Display() {
75          if (Empty()) {
76              std::cout << "Queue is empty\n";
77              return;
78          }
```

```
79          std::cout << "Queue: ";
80          for (int i = 0; i < q_size; i++) {
81              int index = (q_front + i) % q_capacity;
82              std::cout << q_array[index];
83          }
84          std::cout << std::endl;
85      }
86  };
87
88  #endif
```

Main File:

ILOB_2 > mainpt2.cpp > ...

```
1   #include "queueARRchar.h"
2
3   int main() {
4       QueueChar q(10);
5
6       std::cout << "==============================" << std::endl;
7       std::cout << "Is Empty? " << q.Empty() << std::endl;
8       std::cout << "==============================" << std::endl;
9
10      // Enqueue characters to form "NINYA"
11      q.Enqueue('N');
12      q.Enqueue('I');
13      q.Enqueue('N');
14      q.Enqueue('Y');
15      q.Enqueue('A');
16      q.Display();
17
18      std::cout << "==============================" << std::endl;
19
20      std::cout << "Front Element: " << q.Front() << std::endl;
21      std::cout << "Back Element: " << q.Back() << std::endl;
22      std::cout << "Size of Queue: " << q.Size() << std::endl;
23
24      std::cout << "==============================" << std::endl;
25
26      // Pop one character
27      std::cout << "Character to be popped: " << q.Front() << std::endl;
28      q.Dequeue();
29      q.Display();
30
31      std::cout << "==============================" << std::endl;
32
33      // Add another character
34      q.Enqueue('H');
35      std::cout << "After Enqueue 'H': ";
36      q.Display();
37
38      std::cout << "==============================" << std::endl;
39
40      // Clear queue
41      q.Clear();
42      std::cout << "After Clear -> ";
43      q.Display();
44
45      std::cout << "==============================" << std::endl;
46
47      return 0;
48  }
```

Output:

```
==============================
Is Empty? 1
==============================
Queue: NINYA
==============================
Front Element: N
Back Element: A
Size of Queue: 5
==============================
Character to be popped: N
Queue: INYA
==============================
After Enqueue 'H': Queue: INYAH
==============================
After Clear -> Queue is empty
==============================
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Header File (Job):

ILOC > C Job.h > ...

```
1   #ifndef JOB_H
2   #define JOB_H
3
4   #include <iostream>
5   #include <string>
6
7   class Job {
8   public:
9       int id;
10      std::string user;
11      int pages;
12
13      Job(int i = 0, std::string u = "", int p = 0) {
14          id = i;
15          user = u;
16          pages = p;
17      }
18
19      void display() const {
20          std::cout << "Job ID: " << id
21                    << "| User: " << user
22                    << "| Pages: " << pages << std::endl;
23      }
24  };
25
26  #endif
```

Header File (Printer):

ILOC > C Printer.h > ...

```
#ifndef PRINTER_H
#define PRINTER_H

#include "Job.h"

class Printer {
private:
    Job* q_array;
    int q_capacity;
    int q_size;
    int q_front;
    int q_back;

public:
    // Constructor
    Printer(int capacity = 10) {
        q_capacity = capacity;
        q_array = new Job[q_capacity];
        q_size = 0;
        q_front = 0;
        q_back = -1;
    }

    // Destructor
    ~Printer() {
        delete[] q_array;
    }

    // Add new job
    void addJob(Job j) {
        if (q_size == q_capacity) {
            std::cout << "Queue is full! Cannot add job.\n";
            return;
        }
        q_back = (q_back + 1) % q_capacity;
        q_array[q_back] = j;
        q_size++;
        std::cout << "Added Job >> ";
        j.display();
    }
```

```
41
42      // Process next job
43      void processJob() {
44          if (q_size == 0) {
45              std::cout << "No jobs to process.\n";
46              return;
47          }
48          std::cout << "Processing Job >> ";
49          q_array[q_front].display();
50          q_front = (q_front + 1) % q_capacity;
51          q_size--;
52      }
53
54      // Display all jobs in queue
55      void displayQueue() {
56          if (q_size == 0) {
57              std::cout << "Queue is empty.\n";
58              return;
59          }
60          std::cout << "Pending Jobs:\n";
61          for (int i = 0; i < q_size; i++) {
62              int index = (q_front + i) % q_capacity;
63              q_array[index].display();
64          }
65      }
66  };
67
68  #endif
```

Main File:

ILOC > mainC.cpp > ...

```cpp
#include <iostream>
#include "Job.h"
#include "Printer.h"

int main() {
    Printer printer(5);

    std::cout << "------------- Adding Jobs -------------\n";

    // Add jobs
    printer.addJob(Job(1, "Ninya", 5));
    printer.addJob(Job(2, "Joma", 29));
    printer.addJob(Job(3, "Fenz", 8));

    std::cout << "\n------------- Current Printing Queue -------------\n";
    printer.displayQueue();

    // Process jobs one by one
    std::cout << "\n------------- Processing Jobs -------------\n";
    printer.processJob();  // Ninya
    printer.processJob();  // Joma

    std::cout << "\n------------- Queue After Processing -------------\n";
    printer.displayQueue();

    // Adds another job
    printer.addJob(Job(4, "Ella", 20));
    printer.addJob(Job(5, "Sir Jim", 1));

    std::cout << "\n------------- Final Queue -------------\n";
    printer.displayQueue();

    // Process remaining jobs
    std::cout << "\n------------- Processing Remaining Jobs -------------\n";
    printer.processJob();  // Fenz
    printer.processJob();  // Ella
    printer.processJob();  // Sir Jim

    // Checks if the queue is empty after processing all jobs
    std::cout << "\n------------- Queue After All Jobs -------------\n";
    printer.displayQueue();

    return 0;
}
```

Output:

```
------------- Adding Jobs -------------
Added Job >> Job ID: 1| User: Ninya| Pages: 5
Added Job >> Job ID: 2| User: Joma| Pages: 29
Added Job >> Job ID: 3| User: Fenz| Pages: 8

------------- Current Printing Queue -------------
Pending Jobs:
Job ID: 1| User: Ninya| Pages: 5
Job ID: 2| User: Joma| Pages: 29
Job ID: 3| User: Fenz| Pages: 8

------------- Processing Jobs -------------
Processing Job >> Job ID: 1| User: Ninya| Pages: 5
Processing Job >> Job ID: 2| User: Joma| Pages: 29

------------- Queue After Processing -------------
Pending Jobs:
Job ID: 3| User: Fenz| Pages: 8
Added Job >> Job ID: 4| User: Ella| Pages: 20
Added Job >> Job ID: 5| User: Sir Jim| Pages: 1

------------- Final Queue -------------
Pending Jobs:
Job ID: 3| User: Fenz| Pages: 8
Job ID: 4| User: Ella| Pages: 20
Job ID: 5| User: Sir Jim| Pages: 1

------------- Processing Remaining Jobs -------------
Processing Job >> Job ID: 3| User: Fenz| Pages: 8
Processing Job >> Job ID: 4| User: Ella| Pages: 20
Processing Job >> Job ID: 5| User: Sir Jim| Pages: 1

------------- Queue After All Jobs -------------
Queue is empty.
```

Code Analysis:

In my header files for ILO C, I separated the important factors of the code into two clear parts: the Job class and the Printer class. However, instead of a class only I chose to create header files for Job and Printer. The Job.h file is simple, it just defines what a print job looks like. Each job has an ID, the name of the user who submitted it, and the number of pages. I also added a small display() function so that whenever I print a job, it shows all its details in one line. This makes the output easier to read and keeps the job information organized. It makes my

coding easier too as I can track my progress while testing it. In the Printer.h file is where the actual queue logic happens. Inside this header file, I used an array to store the jobs, along with variables to keep track of the capacity, the current size, and the positions of the front and back of the queue. The functions in this header file match the operations of a queue: addJob() puts a new job at the back of the queue (Enqueue), processJob() takes the job at the front and removes it (Dequeue), these operations simulates a printer finishing printing jobs. Further, I added displayQueue(), which loops through the array/queue and shows all the jobs waiting to be printed. By putting these functions in the header file, I made the printer class reusable and easy to test in the main program.

Learnings/Conclusions

In this activity, I learned how queues operate as a fundamental data structure following the FIFO principle, and how they can be implemented using both STL and custom approaches with arrays and linked lists. The procedure highlighted the importance of understanding each operation queues have: enqueue, dequeue, front, back, and clear. Also how they behave differently depending on the internal representation. Using STL made the process straightforward since built-in functions handled most of the complexity, while the linked list and array-based implementations required careful management of pointers, indices, and dynamic memory. The supplementary activity, which simulated a shared printer system, showed me the practical application of queues in real-world scenarios where tasks must be processed in order. This exercise showed how queues ensure fairness and efficiency in resource allocation. The supplementary part made me look back on a past project I had for our Object Oriented Programming. I made a Virtual Queue System for the Student Accounting Office. This makes me wonder if I can use this approach to remake such a system.

Overall, I did an average job in this activity. I was able to follow the procedures and produce working code, but I relied heavily on examples from research and search engines rather than exploring creative or modified solutions of my own. My main area for improvement is developing creativity in implementation like experimenting with variations, adding modifications I want like error handling or visualization, and writing cleaner, more reusable code. By doing so, I can move beyond simply replicating or using memorized codes only and start building more innovatively.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.